

(Within 4 pages. Code is modular and documented in PS9_City_Surveillance_Agent.py)

1. PROBLEM MODELLING

The city is modelled as an undirected, weighted graph $G = (V, E)$:

- V = tourist landmarks (e.g. Marina Beach, Mahabalipuram, ...)
- E = lanes connecting landmarks; weight = distance/cost of the lane.

The drone must traverse EVERY lane exactly once (no lane repeated) while landmarks MAY be revisited. This is the classic Route Inspection / Eulerian traversal problem. A walk covering every edge exactly once exists iff the graph is connected and has either 0 odd-degree vertices (Eulerian circuit) or exactly 2 odd-degree vertices (Eulerian path). Our test map is built with all even degrees, so a closed patrol (start = end) covering all lanes exists.

2. PEAS

Performance measure : Cover all lanes exactly once (no repetition); complete the patrol; minimise revisits/turn-arounds; report real run time and memory.

Environment : City map of landmarks (vertices) and lanes (edges); the drone's current landmark and the set of already-used lanes

Actuators : Drone propulsion/navigation that moves it along a chosen lane from the current landmark to an adjacent one.

Sensors : Position sensor (current landmark), map/topology sensor (adjacent lanes and weights), lane-usage tracker (which lanes are already covered).

3. ENVIRONMENT CHARACTERISTICS

Fully observable : entire map and usage state are known.

Deterministic : moving along a chosen lane has a certain outcome.

Sequential : each move constrains the remaining feasible moves.

Static : map does not change during the patrol.

Discrete : finite landmarks, lanes and actions.

Single agent : one surveillance drone.

Known : transition rules (graph adjacency) are known in advance.

4. HEURISTIC FUNCTION (used by GBFS)

At a landmark, for every candidate neighbour reachable by an UNUSED lane:

$$h(\text{neighbour}) = \frac{\text{number of unused lanes still incident to neighbour} + 1}{1 + \text{number of lanes already used at neighbour}}$$

Greedy Best First Search expands the candidate with the LOWEST h first.

Rationale:

- The first term prefers neighbours that have few unused lanes left, so the agent "finishes off" nearly-complete landmarks before they get stranded (this avoids leaving isolated unused lanes behind).
- The tie-breaker $1/(1+\text{used})$ gently favours less-touched landmarks to keep future options open. h depends only on current state $\Rightarrow$ admissible-style greedy guidance (no path cost $g(n)$ is used, which is what makes it GBFS,

not A*).

5. DATA STRUCTURES

Graph (adjacency list): dict landmark -> list of (neighbour, weight).
add_edge() rejects self-loops/duplicates/empty names (error handling).
Stack (bounded LIFO): used to backtrack along the walk.
- push(): prints "[Stack OVERFLOW] ..." and refuses when full.
- pop() : prints "[Stack UNDERFLOW] ..." and returns None when empty.
Capacity = E + 1 (a trail over E lanes visits at most E+1 landmarks).
visited_edges (set): O(1) test of whether a lane was already used.

6. ALGORITHM (GBFS-guided Eulerian traversal)

1. push(start) onto the Stack.
2. While the Stack is not empty:
 current = peek().
 Among current's UNUSED lanes pick the neighbour with the best (lowest) h(neighbour) -- this is the Greedy Best First choice.
 If such a lane exists: mark it used, push(neighbour), record the trail as an intermediate path.
 Else (dead-end): pop() current and append it to the circuit.
3. Reverse the circuit -> final patrol path.

The explicit Stack splices sub-tours (Hierholzer style) so EVERY lane is guaranteed to be covered without ever re-using a lane.

Pseudocode:

```
push(start)
while stack not empty:
    v = peek()
    e = best_unused_lane_by_heuristic(v)
    if e exists: mark(e); push(other_end(e))
    else: circuit.append(pop())
return reverse(circuit)
```

7. SPACE & TIME COMPLEXITY (MEASURED, not theoretical)

The program measures real values with time.perf_counter() and tracemalloc and prints them to outputPS9.txt, e.g. for the supplied 6-landmark / 9-lane map:

```
Actual execution time : ~0.0001 s
Peak memory allocated : ~4.1 KB
Landmarks explored    : 19
```

(For reference only, the asymptotic cost is $O(V + E)$ time and $O(V + E)$ space, since each lane is pushed/popped once and each landmark stored once.)

8. ALTERNATE MODELLING + PERFORMANCE IMPLICATIONS

Alternate approach: model the task as full STATE-SPACE SEARCH where a state is (current landmark, frozenset of used lanes) and run DFS/backtracking (or A*) until all lanes are used.

- Pros : can optimise secondary objectives (e.g. least total weight) and handles graphs with no Eulerian path by allowing minimal repeats.
- Cons : the state space is exponential -- up to $V * 2^E$ states. Time blows up to $O(2^E)$ and memory to $O(2^E)$ for the visited-state table, so it is impractical beyond ~20 lanes.

Our GBFS + Stack (Hierholzer) approach instead runs in linear $O(V + E)$ time and space and still guarantees full coverage when an Eulerian path exists, which is why it is preferred here. The trade-off: it does not by itself minimise total

weight or repair non-Eulerian graphs.

9. INPUT / OUTPUT

inputPS9.txt (fields are comma-separated, so multi-word landmark names such as "Marina Beach" are supported; plain whitespace also works for single-word names):
line 1 : starting landmark
line 2 : number of lanes E
next E rows: <landmark1>, <landmark2>, <weight>
outputPS9.txt: starting point, coverage check, all intermediate paths, the final path (landmarks joined by " -> "), and the MEASURED time/memory.
Errors handled: missing file, bad lane count, empty/invalid lines, bad weight, start landmark absent from the map, stack overflow/underflow.

End of design document - Group G041
